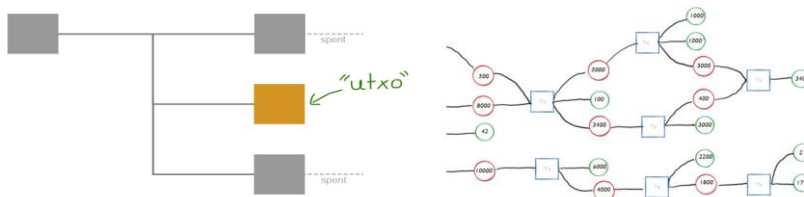


Paper Mixer Prestifilippo

UTXO Model

Un **Unspent Transaction Output** in Bitcoin è un **output** di una **transazione precedente**, che **non è stato ancora speso**. Rappresenta una moneta digitale disponibile per essere utilizzata come **input** in una **nuova** transazione.

In Bitcoin, una transazione non **sposta fondi** da un account all'altro, ma **"consuma" output non spesi** di transazioni **precedenti**, e ne **crea** di nuovi.

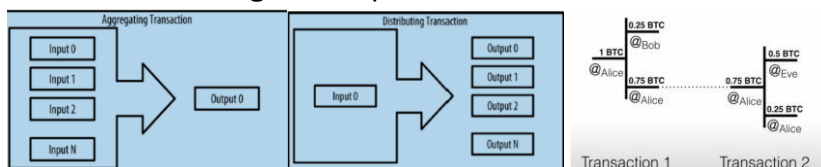


Ogni **input** di una transazione punta a un UTXO che il mittente ha ricevuto in passato e non ha ancora speso. Ogni output di una transazione genera nuovi UTXO, che vengono aggiunti al wallet del destinatario e rappresentano fondi disponibili per future spese. Se un output è stato utilizzato come input in una nuova transazione, viene considerato **"speso"**, e non è più disponibile per ulteriori transazioni. Se **non è ancora stato speso**, è un **UTXO**. Si evita con questo meccanismo il problema del **double spending**.

Funzionamento transazioni: Quando un utente vuole inviare Bitcoin, il suo wallet seleziona **uno o più UTXO** che, sommati, coprono l'importo da inviare. Ogni input contiene il riferimento (ID della transazione precedente e indice) a un UTXO specifico e uno script di sblocco. La transazione crea nuovi UTXO di output. **Uno o più output** vengono destinati al destinatario e, se necessario, uno output di **"cambio"** viene inviato all'utente stesso se il totale degli input supera l'importo da trasferire.

La differenza tra la somma degli input e la somma degli output rappresenta la **commissione per il miner** che includerà la transazione nel blocco.

Tutti gli input usati devono essere **consumati completamente**. **Non possono tornare al mittente**, ma possono essere **trasformati** solo in **output**, verso il destinatario o verso il mittente sotto forma di **change**. Una transazione può essere composta da **M input** e da **N output**. Sulla blockchain viene registrata una catena ininterrotta di transazioni, mostrando come gli UTXO passano di mano in mano.



Proprietà UTXO: Quando un utente A riceve bitcoin come output di una transazione, essi vengono "bloccati" a un suo indirizzo.

Ogni output non speso contiene uno **script** che **vincola** i fondi alla **chiave pubblica** del nuovo proprietario. I fondi possono essere spesi solo da chi **possiede** la **chiave privata** corrispondente alla **chiave pubblica** indicata.

Se il proprietario decide di utilizzare quell'UTXO in una nuova transazione, deve

"sbloccare" i fondi fornendo una **firma digitale** generata con la sua **chiave privata**.

Questa prova, insieme alla chiave pubblica, viene verificata dallo script di sblocco. Se la verifica ha esito positivo, i fondi vengono considerati validamente spesi.



Saldo Bitcoin: Il bilancio di un indirizzo viene calcolato sommando tutti gli UTXO ancora non spesi collegati a tale indirizzo. Se un utente possiede più indirizzi, il saldo totale è la somma dei saldi di ciascun indirizzo.

Problemi di deanomizzazione di Bitcoin e necessità dei mixer

Bitcoin non è un sistema completamente anonimo, ma in realtà offre solo **pseudonimato**. Gli utenti non sono identificati direttamente con nome e cognome, ma attraverso **indirizzi crittografici**. Tuttavia, tutte le transazioni sono **pubbliche** e registrate in modo permanente sulla blockchain. Chiunque può osservare gli indirizzi coinvolti, gli importi trasferiti e la storia completa dei movimenti degli UTXO.

Il **problema principale** è che, anche se un indirizzo Bitcoin non contiene direttamente l'identità reale dell'utente, le transazioni possono comunque essere analizzate, in modo da collegare più indirizzi legati ad uno stesso utente tra loro, e ricostruire il comportamento dell'utente.

Se si ha in una transazione **più indirizzi di input**, è probabile che appartengono alla **stessa persona**.

Un **resto** in una transazione indica un output che **ritorna** al **mittente**. Seguendo quale sia l'indirizzo di resto nelle transazioni, è possibile seguire il denaro di un utente nelle sue future transazioni.

L'**identità dell'utente** può essere rivelata quando un utente compra bitcoin su un exchange che richiede documenti di identità, oppure quando pubblica un proprio indirizzo online.

Una volta stabilito un **collegamento** tra un indirizzo e l'identità di una persona reale, l'analisi della blockchain può permettere di **associare** una **persona** reale alle proprie **transazioni** effettuate on chain con tale indirizzo. Questo rappresenta un grave problema di privacy.

Esempio: Sulla blockchain compare la transazione: **A1 + A2 → Bob + C1**.

L'analista osserva che **A1 e A2 vengono usati insieme come input** nella stessa transazione. Seguendo l'euristica, si può dedurre che **A1 e A2 appartengono alla stessa persona**. In più, la transazione produce un output verso **Bob** e un altro output verso **C1**. Se Bob è il destinatario del pagamento, allora **C1 potrebbe essere l'indirizzo di resto**. Quindi l'analista ipotizza: **A1, A2** e forse **C1** appartengono alla stessa persona. Successivamente compare la transazione 2: **C1 + C2 → Negozio + C3**.

L'analista deduce che anche **C2 e C3** appartengono alla stessa persona.

Infine, l'analista scopre che **A1** è stato usato su un **exchange** con **identità reale**, e la sua identità viene fuori come **Alice**. A questo punto deduce che **A1 = Alice**, e che **A2, C1, C2 e C3** sono collegati ad A1, quindi ad **Alice**.

Mixer

Un **mixer** è un servizio che raccoglie i fondi da più utenti, li mescola tra loro e poi li restituisce verso nuovi indirizzi indicati dagli utenti stessi.

Chi osserva la blockchain vede che molti utenti hanno inviato bitcoin al mixer e che molti output sono usciti dal mixer, ma diventa molto difficile capire quale output corrisponde a quale input. Si vuole **rompere il collegamento diretto tra mittente e destinatario**, ostacolando l'analisi della blockchain.

Mixer centralizzato

Un **mixer centralizzato** è un **intermediario unico esterno** che riceve bitcoin da più utenti, li mescola internamente e li restituisce su nuovi indirizzi forniti dagli utenti, in modo da rendere più difficile tracciare il collegamento tra mittente e destinatario.

Esempio: Supponiamo che Alice, Bob e Carol vogliano mixare 1 BTC. Alice invia 1 BTC al mixer e l'indirizzo A_new. Bob invia 1 BTC al mixer e l'indirizzo B_new. Carol invia 1 BTC al mixer e l'indirizzo C_new. Il mixer invia 1 BTC proveniente da Alice a Carol, 1 BTC proveniente da Bob ad Alice, 1 BTC proveniente da Carol a Bob. Il mixer tiene internamente traccia dei collegamenti tra input e output.

Il **problema principale** è che il cliente deve **fidarsi del mixer**. Il mixer potrebbe rubare i fondi, potrebbe registrare e rivelare chi ha mandato cosa, o potrebbe collaborare con un attaccante.

Es: Alice invia al mixer 1 BTC. Il mixer non invia niente ad Alice, ma invia 1 BTC ad un proprio indirizzo. Oppure il mixer rivela che i fondi di Alice sono stati inviati a Carol e i fondi di Carol a Bob.

Mixcoin

Mixcoin è un **mixer centralizzato** progettato per risolvere il problema che il **mixer rubi i soldi**. Prima che Alice mandi i suoi bitcoin, l'utente si fa dare dal mixer una "promessa crittografica". Il mixer firma con la sua chiave privata una promessa, che dice: "Se Alice mi manda X bitcoin, io manderò Y bitcoin all'indirizzo nuovo B entro un certo tempo". Se il mixer non paga, Alice può pubblicare la promessa firmata, rovinando la reputazione del mixer.

Incentivo sulla reputazione: Se un mixer ruba pochi bitcoin, ma viene smascherato pubblicamente, perderà reputazione e nessuno lo userà più. In questo modo smetterà di guadagnare fee nel tempo, perdendo più ricavi rispetto ai bitcoin rubati.

Protocollo

Si ha un cliente Alice (A), che possiede un certo numero di Bitcoin presso un indirizzo κ_{in} , che assumiamo sia collegabile alla sua identità nel mondo reale. Alice desidera trasferire parte dei suoi fondi a un nuovo indirizzo κ_{out} in modo tale che sia **difficile collegare** κ_{out} a κ_{in} , e quindi ad Alice stessa, in cambio di una commissione di mixing.

Alice invierà alcune delle sue monete a un **mix M**, un'entità centralizzata esterna che manterrà i fondi di Alice in deposito per un periodo di tempo concordato prima di inviare un valore equivalente a κ_{out} .

Problema del furto: Un mixer malevolo potrebbe inviare i fondi di Alice ad un proprio indirizzo segreto κ_M invece che a κ_{out} , come richiesto. La promessa crittografica mira a risolvere questo problema.

Problema deanonimizzazione: Un mixer malevolo potrebbe condividere i propri registri, rivelando che Alice possiede entrambi gli indirizzi κ_{in} e κ_{out} .

Funzionamento

Alice divide il valore totale che vuole mixare in più chunk di dimensione v . Ogni esecuzione del protocollo mescola un singolo "chunk" di dimensione v .

Passo 1: Alice contatta il mix tramite un canale anonimo e propone un insieme di parametri di mixing:

v: Il **valore**, cioè la dimensione del chunk, da mescolare

t1: La **deadline** entro cui **Alice** deve **inviare** i fondi al mix

t2: La **deadline** entro cui il **mix** deve **restituire** i fondi ad Alice

κ_{out} : L'**indirizzo** verso cui Alice desidera **trasferire** i propri fondi

p: Il **tasso** della **commissione** di mixing che Alice pagherà

w: Il numero di **blocchi** che il mix richiede per **confermare** il pagamento di Alice

Passo 2a: Se il mix accetta questi termini, genera un nuovo indirizzo di escrow κ_{esc} e restituisce una garanzia contenente tutti i parametri di Alice più κ_{esc} , firmata usando la propria chiave privata.

Passo 2b: Il mix può rifiutare la richiesta di Alice. Anche Alice non ha alcun obbligo di trasferire i fondi dopo aver ricevuto una garanzia. Se Alice rifiuta, oppure dimentica, di farlo entro la deadline $t1$, il mix può cancellare i propri registri e andare avanti.

Passo 3: Se Alice **trasferisce** effettivamente il valore concordato v a k_{esc} entro la deadline t_1 , allora il **mix** è obbligato a **trasferire** un valore equivalente a k_{out} entro il tempo t_2 .

Passo 4a: Se il mix lo **fa fedelmente**, allora entrambe le parti dovrebbero **distruggere** i propri registri per garantire anonimato futuro contro eventuali violazioni dei dati.

Passo 4b: Se invece il mix **non riesce** a trasferire il valore v a k_{out} **entro** il tempo t_2 , allora **Alice pubblica** la propria **garanzia**. Poiché la garanzia è firmata dalla chiave privata del mixer e tutte le transazioni Bitcoin sono registrate pubblicamente, chiunque può **verificare** che il mix abbia **imbrogliato**.

Esempio semplice di una singola esecuzione

Alice vuole mixare **1 BTC**. Prima contatta il mixer usando un canale anonimo, dicendo: "Voglio mixare 1 BTC. Ti mando i fondi entro le 12:00. Tu devi restituirmi 1 BTC, meno la commissione, entro le 15:00. Devi mandarli a questo nuovo indirizzo: k_{out} . La commissione è dell'1%. Voglio che tu aspetti 6 blocchi prima di considerare valido il mio pagamento."

Il mixer accetta. Allora genera un nuovo indirizzo k_{esc} , dove Alice deve mandare il suo 1 BTC.

Poi il mixer firma una garanzia, che dice: "Io, mixer, prometto che se Alice manda 1 BTC all'indirizzo k_{esc} entro le 12:00, allora io manderò 0.99 BTC all'indirizzo k_{out} entro le 15:00." A questo punto Alice manda **1 BTC** all'indirizzo k_{esc} prima delle 12:00. Dopo che la transazione riceve **6 conferme**, il mixer deve rispettare l'accordo e mandare **0.99 BTC** all'indirizzo nuovo di Alice, cioè k_{out} , entro le 15:00.

Se il mixer si comporta correttamente, Alice riceve i bitcoin su un nuovo indirizzo.

Se invece il mixer non manda i 0.99 BTC entro le 15:00, Alice pubblica la garanzia firmata dal mixer, rovinando la sua reputazione.

Accortezze e criticità

La privacy aumenta se molti utenti usano lo stesso mixer, gli importi sono simili, i pagamenti avvengono con ritardi diversi, gli utenti non riutilizzano indirizzi e il mixer non rivela i collegamenti interni.

Mixcoin consente di rilevare il furto, ma non lo previene. Se il mixer decide di non restituire i fondi, l'utente può denunciarlo pubblicamente, ma non può impedire materialmente il furto.

Un'altra criticità importante è che il mixer conosce il collegamento tra mittente e destinatario. Infatti l'utente deve comunicare al mixer sia l'indirizzo da cui partono i fondi sia l'indirizzo su cui vuole riceverli. Di conseguenza, il mixer può memorizzare questi dati, ricostruire la relazione tra input e output e potenzialmente rivelarla.

CoinShuffle: Practical Decentralized Coin Mixing for Bitcoin

CoinShuffle

Mixing decentralizzato: Protocollo di mixing utilizzato per mescolare i bitcoin di più utenti **senza** usare un **mixer centrale** esterno. Gli utenti collaborano direttamente tra loro per costruire una **transazione comune**, in cui osservando la blockchain, non sia facile capire quale input appartiene a quale output.

CoinShuffle è un mixer decentralizzato basato su CoinJoin. Viene utilizzata la proprietà di Bitcoin per cui è possibile creare una transazione con **più input e più output**. Gli utenti costruiscono una transazione unica, in cui i fondi vengono inviati da ogni membro dal proprio indirizzo di input ai loro nuovi indirizzi di output, in modo mescolato. Tutti gli input sono visibili e tutti gli output sono visibili sulla blockchain, ma non è chiaro quale output appartenga a quale membro.

Input Alice ---> Output X

Input Bob ---> Output Y

Input Charlie ---> Output Z

In caso di tentativo di **frode**, l'utente semplicemente rifiuterà di firmare la transazione.

Anonymity set: Gruppo dentro cui un utente si nasconde, formato da tutti i partecipanti della transazione comune. Ogni output potrebbe appartenere ad uno qualsiasi dei membri del anonymity set.

Problema: Se ogni membro **manda** semplicemente il proprio **nuovo indirizzo** a tutti gli altri membri in broadcast, allora il collegamento è **noto** ai **partecipanti** stessi. La transazione finale può anche sembrare anonima sulla blockchain, ma i **partecipanti** **sanno** comunque chi **controlla** quale **output**.

Protocollo

Si suppone che ogni partecipante detenga la stessa quantità di monete presso un certo indirizzo Bitcoin. Tale indirizzo viene utilizzato come input nella transazione di mixing.

Announcement: Ogni partecipante genera una nuova **coppia di chiavi** pubblica e privata, e **trasmette** in broadcast la **chiave pubblica** di cifratura generata. Queste chiavi serviranno nella fase successiva per cifrare e decifrare gli indirizzi di output.

Shuffling: Ogni partecipante crea un **nuovo indirizzo** Bitcoin, usato come indirizzo di output nella transazione di mixing. I partecipanti mescolano gli indirizzi di output appena generati nel seguente modo. I partecipanti sono ordinati.

Il **primo partecipante** prende il proprio **indirizzo di output** e lo **cifra** a strati usando le **chiavi pubbliche** dei **partecipanti successivi**, $c_1 = [Enc((pubKey_2, \dots, pubKey_n))]$. Poi firma questa lista, $[Sig(privateKey_1, (C_1))]$, e la invia al secondo partecipante.

Quando un **partecipante intermedio** ($i \in \{2, \dots, N-1\}$) riceve la lista, rimuove da ogni elemento presente lo strato di cifratura che può aprire con la propria chiave privata. Non necessariamente vede gli indirizzi finali, perché possono rimanere cifrati con le chiavi dei partecipanti successivi. Dopo questo, aggiunge alla lista anche il **proprio indirizzo** di output **addressOut_i**, cifrandolo con le **chiavi pubbliche** dei **partecipanti** che vengono **dopo** di lui, ottenendo **Enc((pubKey_{i+1}, ..., pubKey_n), addressOut_i)**. Infine **mescola** casualmente **l'ordine della lista**, $C_i = \text{shuffle}([c_{\pi 1}, \dots, c_{\pi n}])$, la firma e la **invia** al partecipante **successivo** $i+1$.

Se una decifratura fallisce o se due operazioni di decifratura portano allo stesso output, il partecipante i entra nella fase di blame.

Fase 3: Broadcast dell'output

Dopo aver ricevuto l'ultimo σ , il partecipante N rimuove l'ultimo strato di cifratura da ogni ciphertext nel vettore C_{N-1} . Poi il partecipante N mescola il vettore risultante di indirizzi di output dopo averlo esteso con il proprio indirizzo di output **addressOut_N**, ottenendo il vettore finale **Tout**. Infine, il partecipante N trasmette in broadcast $\sigma_N = Sig(privateKey_N, (C_N))$ al resto dei partecipanti. Se il protocollo è stato eseguito correttamente da tutti i partecipanti, a questo punto ogni partecipante ha ricevuto una copia del vettore mescolato **Tout** degli indirizzi di output. Ogni partecipante i controlla se il proprio indirizzo di output **addressOut_i** è contenuto in **Tout**.

Se lo trova, il protocollo può continuare. Se invece il suo indirizzo manca, entra nella fase di blame.

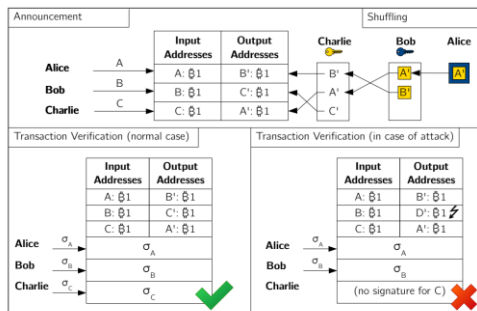
Fase 4: Controllo di equivocation

Si controlla che tutti abbiano ricevuto dagli altri partecipanti la stessa lista di chiavi pubbliche e la stessa lista di indirizzi di output, durante uno dei broadcast. Per assicurarsi di questo, ogni partecipante i calcola $h_i = Hash((pubKey_2, \dots, pubKey_n), Tout)$ e trasmette in broadcast $\sigma_i = Sig(privateKey_i, (h_i))$.

Ogni partecipante i controlla se ci sono due partecipanti a e b con $h_a \neq h_b$. In tal caso, il partecipante i entra nella fase di blame.

Transaction Verification. Ogni partecipante verifica che il proprio indirizzo di output si trova effettivamente nella lista TOut. In caso positivo, ogni partecipante crea una transazione di mixing, non ancora firmata, che spende monete da tutti gli indirizzi di input e le invia alla lista mescolata di indirizzi di output. Ogni partecipante **firma** la **transazione** con la propria **chiave privata** Bitcoin e trasmette in **broadcast** la **firma**. Dopo aver ricevuto le firme da tutti gli altri partecipanti, ogni partecipante è in grado di creare una **versione** completamente **firmata** della **transazione** di mixing, che può essere inviata su **Bitcoin**.

Blame. In ogni passo delle fasi precedenti, ogni partecipante si assicura che nessun altro partecipante devii dal protocollo. In tal caso, viene segnalata la deviazione e si identifica il partecipante che si è comportato in modo scorretto. Il partecipante scorretto può quindi essere **escluso** da una successiva esecuzione del protocollo.



Esempio completo di CoinShuffle con tutte le fasi

Si hanno 3 partecipanti, Alice, Bob e Charlie. Alice ha come indirizzo di input A_old con 1 BTC. Bob usa B_old con 1 BTC. Charlie usa C_old con 1 BTC. Ognuno vuole mixare **1 BTC** e ricevere i fondi sui nuovi indirizzi A_new, B_new e C_new.

Fase 1: Announcement

I partecipanti scelgono il seguente ordine: 1. Alice, 2. Bob, 3. Charlie.

Poi Bob e Charlie generano una **coppia temporanea** di **chiavi** di cifratura. Bob genera: **pubKeyBob**, **privKeyBob**. Charlie genera: **pubKeyCharlie**, **privKeyCharlie**. pubKeyBob e pubKeyCharlie vengono pubblicate.

Alice, Bob e Charlie controllano che tutti abbiano davvero almeno **1 BTC** negli indirizzi di input.

Fase 2: Shuffling

Si ha che $\text{Enc}(\text{pubKeyCharlie}, A_{\text{new}})$, significa: A_new è chiuso in una busta che solo Charlie può aprire con la propria privKeyCharlie.

Se scrivo $\text{Enc}(\text{pubKeyBob}, \text{Enc}(\text{pubKeyCharlie}, A_{\text{new}}))$ significa A_new è chiuso prima nella busta di Charlie, poi quella busta è chiusa nella busta di Bob.

Passo 2.1: Alice inserisce il suo indirizzo

Alice cifra il suo indirizzo con le chiavi dei partecipanti successivi, cioè Bob e Charlie:
 $c_A = \text{Enc}(\text{pubKeyBob}, \text{Enc}(\text{pubKeyCharlie}, A_{\text{new}}))$

Alice manda a Bob la lista: $[\text{Enc}(\text{pubKeyBob}, \text{Enc}(\text{pubKeyCharlie}, A_{\text{new}}))]$.

Passo 2.2: Bob rimuove uno strato, aggiunge il suo indirizzo e mescola

Bob riceve la lista cifrata. Può togliere solo lo strato cifrato con la sua chiave pubblica, perché possiede **privKeyBob**.

Dopo la decifratura ottiene: $[\text{Enc}(\text{pubKeyCharlie}, A_{\text{new}})]$. Bob **non vede ancora A_new**.

Ora Bob deve aggiungere il suo nuovo indirizzo **B_new**. Dopo Bob c'è ancora Charlie, quindi Bob cifra B_new con la chiave pubblica di Charlie: $c_B = \text{Enc}(\text{pubKeyCharlie}, B_{\text{new}})$.

Ora Bob ha due elementi: $[\text{Enc}(\text{pubKeyCharlie}, A_{\text{new}}), \text{Enc}(\text{pubKeyCharlie}, B_{\text{new}})]$

Prima di passarli a Charlie, Bob li mescola casualmente, ottenendo **[Enc(pubKeyCharlie, B_new), Enc(pubKeyCharlie, A_new)]**.

Passo 2.3: Charlie rimuove l'ultimo strato, aggiunge il suo indirizzo e mescola

Charlie riceve la lista da Bob. Charlie possiede **privKeyCharlie**, quindi può decifrare entrambi, ottenendo **[B_new, A_new]**.

Charlie non sa quale dei due era originariamente quello di Alice e quale è quello di Bob, perché Bob ha mescolato l'ordine.

Charlie aggiunge alla lista anche il suo indirizzo **C_new**. La lista diventa: **[B_new, A_new, C_new]**.

Poi Charlie **mescola** ancora una volta l'ordine. Ottiene: **Tout = [C_new, A_new, B_new]**. Charlie infine manda in **broadcast** la **lista Tout** a tutti.

Fase 3: Transaction Verification

Ogni partecipante controlla che il proprio indirizzo sia presente nella lista **Tout**, rispettivamente A_new, B_new, C_new. Se un partecipante **non trova** il proprio indirizzo, **non firma** e si va nella **fase di blame**.

In questo caso tutto va bene e ogni partecipante costruisce la transazione di mix non ancora firmata.

Input:

A_old 1 BTC → C_new

B_old 1 BTC → A_new

C_old 1 BTC → B_new

Fase 4: Controllo di equivocation

Qualche partecipante potrebbe avere inviato a qualche altro una chiave scorretta.

Charlie potrebbe provare a mandare ad Alice la lista corretta: **[C_new, A_new, B_new]** e a Bob un'altra lista malevola: **[C_new, X_fake, B_new]**.

Ogni partecipante calcola **h = H(chiavi pubbliche, Tout)**. Se tutti hanno ricevuto la **stessa lista** e le **stesse chiavi**, tutti calcolano lo **stesso hash**.

Alice calcola: h1 = 12345. Bob calcola: h2 = 12345. Charlie calcola: h3 = 12345. Tutto ok. Se qualcuno avesse ricevuto dati diversi, si avrebbe avuto uno scenario del tipo:

Alice calcola: h1 = 12345. Bob calcola: h2 = 99999. Charlie calcola: h3 = 12345. In tal caso qualcuno avrebbe fatto inviato versioni diverse del protocollo a partecipanti diversi. In quel caso si entra nella fase di blame.

Fase 5: Firma e invio della transazione

Se tutti i controlli vanno bene, ogni partecipante firma il proprio input nella transazione di mix in modo da spenderlo, rispettivamente A_old, B_old, C_old.

Quando tutte le firme sono raccolte, la transazione è completa e può essere pubblicata sulla rete Bitcoin.

Fase 6: Blame

Se per esempio Bob non ha davvero 1 BTC. Bob interrompe il protocollo. Charlie manda liste diverse a persone diverse. Qualcuno modifica la lista degli output. Qualcuno non firma alla fine. In questi casi i partecipanti pubblicano le informazioni necessarie per ricostruire cosa è successo, ed escludono il colpevole dalle prossime esecuzioni.

Criticità

L'anonimato dipende dal numero di partecipanti. Se partecipano pochi utenti, l'anonymity set è piccolo.

Gli importi devono essere tutti uguali. Se gli importi sono diversi, si possono fare collegamenti tramite analisi delle quantità.

Le transazioni CoinShuffle possono essere ben riconoscibili, avendo molti output dello stesso valore. Un osservatore può capire che quella è una transazione di mixing, e un utente non può negare di averci partecipato.

Un partecipante può partecipare alla fase iniziale e poi rifiutarsi di firmare alla fine. In questo caso la transazione non può essere completata, facendo perdere tempo agli altri, portando ad un DoS.

Infine, durante la fase di shuffling l'ultimo utente controlla la lista mescolata finale e potrebbe riordinare gli indirizzi in un modo che gli dà un vantaggio, per esempio cercando di collegare un input preferito al proprio indirizzo di output.

CoinSwap

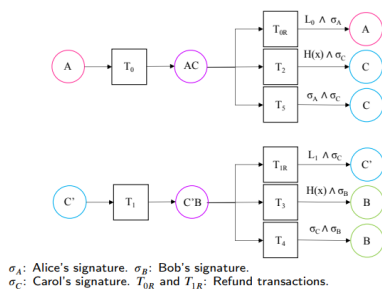
Problema: Alice vuole pagare Bob, ma non vuole che sulla blockchain si capisca facilmente che il pagamento parte da lei e arriva a Bob.

Per risolvere questo problema entra in gioco Carol, che fa da **intermediaria**. Alice dà delle monete a Carol, e Carol paga Bob usando altre monete, non collegate direttamente a quelle di Alice.

Problema: Carol potrebbe prendere le monete di Alice e non pagare Bob. Oppure Bob potrebbe ricevere il pagamento e Carol potrebbe non riuscire a recuperare i soldi da Alice.

Transazione multisignature 2-of-2: Transazione in cui i fondi possono essere spesi solo se firmano entrambe le parti coinvolte.

HashLock: Bob sceglie un valore segreto X , calcola il suo hash $H(X)$, e lo comunica a Carol e Alice, senza rivelare X .



T0 - Alice blocca i suoi fondi con Carol: Alice crea una transazione multi-signature 2-of-2 verso AC. I fondi possono essere spesi solo con la firma di Alice e Carol: $\sigma_A \wedge \sigma_C$.

T1 - Carol blocca i suoi fondi con Bob: Carol crea una transazione multi-signature 2-of-2 verso C'B. I fondi possono essere spesi solo con la firma di Carol e Bob: $\sigma_C \wedge \sigma_B$.

T0R – Recupero dei fondi dopo timeout da Carol ad Alice: Alice e Carol preparano una transazione per cui Alice recupera i fondi precedentemente bloccati con Carol in T0, dopo un certo timelock.

T1R – Recupero dei fondi dopo timeout da Bob a Carol: Bob e Carol preparano una transazione per cui Carol recupera i fondi precedentemente bloccati con Bob in T1, dopo un certo timelock, più corto di T0R.

T2 — hashlock per pagare Carol in caso di problemi: Alice prepara una transazione che permette a Carol di prendere i fondi di T0 se Carol conosce il segreto x : $H(x) \wedge \sigma_C$.

T3 — hashlock per pagare Bob in caso di problemi: Carol prepara una transazione che permette a Bob di prendere i fondi di T1 se Bob rivela il segreto x : $H(x) \wedge \sigma_B$.

T4 - Carol sblocca i suoi fondi con Bob: Dopo aver ricevuto X , Carol firma con Bob e sblocca i fondi verso Bob, bloccati precedentemente con T1. Serve la firma di Carol e Bob: $\sigma_C \wedge \sigma_B$.

T5 - Alice sblocca i suoi fondi con Carol: Dopo aver ricevuto X , Alice firma con Carol e sblocca i fondi verso Carol, bloccati precedentemente con T0. Serve la firma di Alice e Carol: $\sigma_A \wedge \sigma_C$.

Se tutto va bene, Carol sblocca i fondi a Bob tramite T4. Dopo aver ricevuto i fondi, Bob invia X a Carol. Dopo che vede che Bob è stato pagato, Alice crea la transazione che sblocca il pagamento a Carol con T5.

Se Bob non viene pagato, usa il segreto X per venir pagato da Carol via T3.

Se Carol non viene pagata da Alice dopo aver pagato Bob, usa il segreto X ricevuto da Bob dopo il pagamento per ottenere i fondi via T2.

Se Bob non vuole dare il segreto X a Carol, dopo un certo timelock Carol recupera i soldi bloccati con Bob via T1R.

Se Carol non paga Bob e non fornisce la prova X ad Alice, dopo un certo timelock, Alice recupera i soldi che aveva bloccato insieme a Carol utilizzando T0R.

Criticità

Nonostante evita il furto, l'intermediario Carol può ancora memorizzare che è avvenuta una transazione mixata tra Alice e Bob.

Il protocollo richiede più transazioni rispetto ad un pagamento normale, aumentando così le commissioni.

Se on-chain si hanno in modo ravvicinato due transazioni multisignature che inviano lo stesso valore, possono essere collegate. Inoltre, quando viene pubblicato il segreto X, il collegamento tra Alice e Bob viene reso evidente.

CoinParty

CoinParty è una tecnica di mixing per Bitcoin pensata per ottenere privacy senza affidarsi a un mixer centrale. Ogni utente parte da un indirizzo $Input_i$, vuole ricevere su un nuovo indirizzo $Output_i$ una quantità v di bitcoin, e il protocollo deve creare una permutazione segreta $Output_{\pi(i)}$ così che input e output non siano collegabili.

CoinParty non usa un singolo server fidato, ma utilizza un gruppo di nodi chiamati **mixing peers**, che collaborano per mescolare gli indirizzi di output e creare una firma ECDSA distribuita, in modo che nessun singolo nodo possieda la chiave privata completa e nessun singolo nodo possa ingannare durante la procedura di shuffling. Si vuole garantire così che i bitcoin non vengano rubati dai mixing peers, che nessuno possa collegare un indirizzo di input i al corrispettivo di output i , e che i peer di input possano negare di aver partecipato al mixing.

Rispetto a un mixer centralizzato, non si ha un singolo ente che può rubare i fondi o conoscere tutta la mappatura input-output. Rispetto a CoinShuffle, non usa una grande transazione di gruppo facilmente riconoscibile, ma usa invece tante transazioni più piccole e apparentemente normali, in modo che un utente possa negare plausibilmente che quella transazione fosse parte di un mixing.

CoinParty viene eseguito in 4 fasi: Commitment, Shuffling, Transaction ed Error.

Fase di Commitment

Nella fase di **commitment** ogni utente i che vuole mixare i propri fondi, li blocca nell'indirizzo di **escrow T_i** .

L'indirizzo di escrow T_i non deve essere controllato da un singolo peer di mixing, perché un peer malevolo potrebbe rubare i fondi.

L'indirizzo di escrow T_i viene **generato** a partire da una **chiave privata** che nessun peer conosce interamente, ma di cui ogni peer ne conosce una parte. In questo modo per spendere i fondi depositati in T_i , non basta un singolo peer, ma deve collaborare una maggioranza dei peer di mixing, così da produrre una firma valida.

L'indirizzo di escrow T_i viene generato in modo distribuito nel seguente modo:

Inizialmente, ogni **peer di mixing M_j** ottiene una **parte** di una **chiave privata $privShareKey_i$** chiamata **share**, generata casualmente. Ogni peer calcola localmente la propria parte della chiave pubblica **pubShareKey_{ij}**.

Ogni peer di mixing invia in broadcast agli altri peer la propria parte della chiave pubblica. Ogni peer ricostruisce localmente tramite interpolazione di Lagrange la chiave pubblica completa **pubShareKey_i**.

Questa chiave pubblica viene hashata per generare **l'indirizzo di escrow T_i**.

Se tutti i peer di mixing comunicano lo stesso indirizzo di escrow T_i, il peer di input trasferisce la quantità richiesta **v** dal proprio indirizzo di input Input_i all'indirizzo di escrow T_i: $I_i \xrightarrow{v} T_i$.

I peer di mixing attendono sei blocchi prima di considerare il commitment valido e accettato dalla rete Bitcoin.

Se invece l'utente i riceve da due peer due indirizzi di escrow T_i e T_{i'} diversi, qualcuno potrebbe aver fatto equivocation, nel tentativo di deviare i fondi in un indirizzo diverso. L'utente i interrompe il protocollo e avvisa gli altri.

Fase di Shuffling

Fase in cui si mescolano gli indirizzi di output forniti dai peer di input, in modo che nessuno possa capire quale indirizzo di input Input_i corrisponde a quale indirizzo di output Output_i. La permutazione segreta degli indirizzi di output viene costruita dai peer di mixing, in modo tale che nemmeno loro sappiano il collegamento finale tra input e output. La permutazione viene costruita nel seguente modo.

Inizialmente, ogni peer di input cifra il proprio indirizzo di output Output_i usando le chiavi pubbliche dei peer di mixing (pubKey_1, pubKey_2, ..., pubKey_m), con una cifratura a strati.

$$\begin{aligned} \llbracket \text{Output}_i \rrbracket_{\text{pubKey}_1:\text{pubKey}_m} \\ = \text{Encrypt}_{\text{pubKey}_1}(\text{Encrypt}_{\text{pubKey}_2}(\dots \text{Encrypt}_{\text{pubKey}_m}(\text{Output}_i))) \end{aligned}$$

Ogni peer di mixing potrà rimuovere solo il proprio strato di cifratura, ma non potrà vedere subito l'indirizzo originale.

Inoltre, ogni peer di input calcola anche l'hash del proprio indirizzo di output (H(Output_i)), e ne invia una sottoparte diversa ad ogni peer di mixing.

Successivamente, i peer iniziano il processo di decifratura e mescolamento.

Il primo peer di mixing M1 rimuove il primo strato di cifratura dagli indirizzi ricevuti, ottenendo per ogni indirizzo di output i:

$$(\text{Encrypt}_{\text{pubKey}_2}(\dots \text{Encrypt}_{\text{pubKey}_m}(\text{Output}_i))).$$

Poi riordina segretamente gli indirizzi cifrati, e passa il risultato al peer successivo.

I peer successivi j eseguono lo stesso procedimento: rimuove il proprio strato di cifratura, applica una propria permutazione privata (π_j), e invia il risultato al peer successivo. $\text{permutedCiphred}^j = \llbracket O_{\pi_j 1} \rrbracket_{\text{pubKey}_{j+1}:\text{pubKey}_m}, \dots, \llbracket O_{\pi_j n} \rrbracket_{\text{pubKey}_{j+1}:\text{pubKey}_m}$

Alla fine, l'ultimo peer di mixing M_m rimuove l'ultimo strato di cifratura, ottenendo gli indirizzi visibili, ma già riordinati molte volte. L'ultimo peer riordina gli indirizzi in **ordine lessicografico** e trasmette il risultato finale agli altri peer di mixing.

Tramite gli share hash ricevuti precedentemente, i peer di mixing controllano che nessun indirizzo sia stato eliminato, modificato o aggiunto durante lo shuffling.

Ogni peer di mixing effettua la somma di tutti gli sharehash ricevuti. Poi ognuno condivide il risultato della somma ed ognuno esegue una somma totale, creando un checksum. Si ricrea poi un checksum con gli hash iniziali generati dai peer di input, sommandoli. Se i checksum sono uguali, significa che gli indirizzi finali sono gli stessi, permutati in ordine diverso.

$$\sum_{i=1}^n H(O_{\pi_m \circ \dots \circ \pi_1(i)}) == \sum_{i=1}^n H(O_i)$$

Si controlla poi che la lista degli indirizzi sia ordinata lessicograficamente. Se uno dei due controlli fallisce, il protocollo entra nella fase di errore e reversione.

Se il controllo ha successo, i peer di mixing usano il checksum per inizializzare un generatore pseudo-casuale, che produce una permutazione finale comune degli indirizzi: $O_{\pi, m+1(1)}, \dots, O_{\pi, m+1(n)}$

Fase di Transazione

Nella fase di **transazione**, i peer di mixing devono trasferire i fondi dagli indirizzi temporanei di escrow T_i agli indirizzi di output mescolati $Output_{\pi(i)}$: $T_i \xrightarrow{v} Output_{\pi(i)}$. Per poter spendere i fondi dall'indirizzo di escrow T_i serve una firma valida, ma la chiave privata è distribuita tra i vari peer di mixing. La firma viene quindi prodotta mettendo insieme le sottoparti di chiave privata dei vari peer di mixing, senza che la chiave privata venga mai ricostruita da un singolo peer, utilizzando la firma ECDSA Treshold. Una volta generata la firma, viene eseguita la transazione.

Fase di Errore e reversione

Se viene rilevato un errore nella fase di commitment o nella fase di shuffling, il mixing viene interrotto. In questo caso, i peer di mixing non completano il trasferimento verso gli indirizzi di output, ma restituiscono i fondi dagli indirizzi di escrow T_i agli indirizzi di input originali Inputi. Per effettuare le transazioni di reversione viene usata ancora la firma distribuita ECDSA Treshold.

I peer di input poi le informazioni relative alla cifratura a strati degli indirizzi $[[Output_i]]_{pubKey_1:pubKey_m}$. I peer di mixing onesti ricostruiscono i passaggi intermedi dello shuffling per capire in quale punto è avvenuta la manipolazione e quale peer è stato.

Criticità

Il protocollo è sicuro solo se 2/3 dei peer di mixing sono onesti.

Poiché i peer di mixing **non hanno fondi propri coinvolti nelle transazioni di escrow**, potrebbero abbandonare il protocollo mentre le monete si trovano ancora negli indirizzi di escrow, provocando reversione e quindi ritardo.

Per incentivare i peer di mixing a partecipare correttamente, si possono introdurre commissioni di mixing, ma questo aumenta il costo per i peer di input.